PRACTICAL-14

PART A:

1. Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.

CODE:

Model/item.java:

```
package lab14.model;


public class Item {
private String name;


    public Item(String name) {
this.name = name;
    }


    public String getName() {
return name;
    }
}
```

Service/ItemService.java:

```
package lab14.service;
```

```
import lab14.model.Item;
import java.util.ArrayList;
public class ItemService {
private ArrayList<Item> items
= new ArrayList<>();

   public void addItem(String name) {        if (name == null ||
name.trim().isEmpty()) {            throw new
IllegalArgumentException("Invalid item name");
       }
       items.add(new Item(name));
   }
   public int getCount() {
return items.size();
   }

   public void performAction(int choice, String name) {        switch
(choice) {            case 1 -> addItem(name);            case 2 ->
System.out.println("Total items: " + getCount());            default -> throw
new IllegalArgumentException("Invalid action");
       }
   }
}
```

ItemServiceTest.java:

```
package lab14;
```

```
import org.junit.jupiter.api.Test;
import lab14.service.ItemService;
import static
org.junit.jupiter.api.Assertions.*;
public class ItemServiceTest {
   @Test     void addingItemIncreasesCount()
{        ItemService service = new
ItemService();

        service.addItem("Book");

        assertEquals(1, service.getCount());
   }
   @Test     void
addingMultipleItemsIncreasesCount() {
ItemService service = new ItemService();

        service.addItem("Pen");
service.addItem("Pencil");

        assertEquals(2, service.getCount());
   }
   @Test     void invalidActionThrowsException() {
ItemService service = new ItemService();
assertThrows(IllegalArgumentException.class,
                () -> service.performAction(9, ""));
```

```
    }
}
```

Main.java: package lab14; import lab14.service.ItemService; import java.util.Scanner; public class Main {

```
public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
ItemService service = new ItemService();
while (true) {
            System.out.println("\n1. Add Item");
            System.out.println("2. Show Count");
            System.out.println("3. Exit");
System.out.print("Enter choice: ");
int choice = sc.nextInt();
sc.nextLine();

            try {
switch (choice) {
case 1:
                        System.out.print("Enter item name: ");
String name = sc.nextLine();
service.performAction(1, name);
                        System.out.println("Item added successfully");
                        break;
```

```
                case 2:
service.performAction(2, "");
break;

                case 3:
                    System.out.println("Exiting...");
sc.close();                     return;

                default:
                    service.performAction(choice, "");
                }
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

OUTPUT:

```
1. Add Item
2. Show Count
3. Exit
Enter choice: 1
Enter item name: chocolate
Item added successfully

1. Add Item
2. Show Count
3. Exit
Enter choice: 2
Total items: 1

1. Add Item
2. Show Count
3. Exit
Enter choice: 3
Exiting...
PS C:\Users\Krish\OneDrive\Documents\JAVA-practicals>
```

LEARNING OUTCOMES:

- To understand the organization of a Java application into model, service, and main classes.
- To implement data management using ArrayList and exception handling.
- To write and execute JUnit test cases using assertions such as assertEquals() and assertThrows().

2. Organise that app into packages, build a runnable JAR, and add a test that saves and reloads its data unchanged.

CODE:

ItemServiceTest.java: package

lab14.test; import

lab14.src.service.ItemService; import

org.junit.jupiter.api.Test; import

java.nio.file.Files; import

java.nio.file.Path;

```
import static org.junit.jupiter.api.Assertions.*;
public class ItemServiceTest {

   @Test     void addingItemIncreasesCount()
{        ItemService service = new
ItemService();         service.addItem("Book");

       assertEquals(1, service.getCount());
   }

   @Test     void
invalidItemThrowsException() {
ItemService service = new ItemService();

       assertThrows(IllegalArgumentException.class,
             () -> service.addItem(""));
   }
```

```
    @Test     void saveAndReloadPreservesData() throws
Exception {         Path file = Files.createTempFile("items",
".dat");


        try {
            ItemService original = new ItemService();
original.addItem("Book");
original.addItem("Pen");


            original.save(file.toString());
            ItemService restored = new ItemService();
restored.load(file.toString());
assertEquals(original.getCount(), restored.getCount());
assertEquals(                   original.getItems().toString(),
restored.getItems().toString()
            );
        } finally {
            Files.deleteIfExists(file);
        }
    }
}
```

Main.java:

```
package lab14.src; import
lab14.src.service.ItemService; import
java.util.Scanner;
```

```
public class Main {     public static void
main(String[] args) {
        Scanner sc = new Scanner(System.in);
ItemService service = new ItemService();
while (true) {
            System.out.println("\n1. Add Item");
            System.out.println("2. Show Items");
            System.out.println("3. Save Items");
            System.out.println("4. Load Items");
            System.out.println("5. Exit");
System.out.print("Enter choice: ");
int choice = sc.nextInt();
sc.nextLine();            try {
switch (choice) {                   case 1:
                    System.out.print("Enter item name: ");
service.addItem(sc.nextLine());
System.out.println("Item added.");
break;                  case 2:
                    System.out.println(service.getItems());
break;                  case 3:
                    service.save("items.dat");
System.out.println("Data saved.");
break;                  case 4:
                    service.load("items.dat");
                    System.out.println("Data loaded.");
break;                  case 5:
```

```
sc.close();                    return;
default:
                    throw new IllegalArgumentException("Invalid choice");
                }
            } catch (Exception e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}
```

Manifest.txt:

Main-Class: lab14.src.Main

OUTPUT:

```
PS C:\Users\Krish\OneDrive\Documents\JAVA-practicals\lab14> java -jar lab14.jar
1. Add Item
2. Show Items
3. Save Items
4. Load Items
5. Exit
Enter choice: 1
Enter item name: bag
Item added.

1. Add Item
2. Show Items
3. Save Items
4. Load Items
5. Exit
Enter choice: 3
Data saved.

1. Add Item
2. Show Items
3. Save Items
4. Load Items
5. Exit
Enter choice: 4
Data loaded.
```

LEARNING OUTCOMES:

- To organize Java classes into packages for better modularity and maintainability.
- To create and execute a runnable JAR file using a manifest file.
- To implement data serialization and deserialization and verify data integrity using JUnit tests.